SoulAI

Plataforma gamificada de missões, avatar e hábitos

Documentação Técnica do Projeto — Sprint 4 (Domain Driven Design using Java)

Nome da solução: SoulAI
Nome da equipe: LLMY
Integrantes:
Lena Haidar Halawi RM: 572258
Lyvia Correia de Amorim RM: 569851
Manuela de Lima Ramos RM: 572956
Yasmin Souza Silva Martins RM: 572102

Turma: 1TDSPF
Disciplina: Domain Driven Design Using Java
Ano: 2026

São Paulo 2026

1. Objetivo e Escopo do Projeto **3**
1.1 Objetivo do Projeto 3
1.2 Escopo do Projeto 3
2. Solução proposta **4**
3. Funcionalidades **4**
4. Tabela de endpoints **6**
Usuário 6
Administrador 7
Avatar 7
Acessório 8
Missão 9
Missões do usuário 9
Inventário do usuário 10
5.Protótipo do Sistema **11**
5.1 Tela Inicial 11
5.2 Tela de Cadastro 12
5.3 Tela de Login 13
5.4 Menu Principal 14
5.5 Tela de Missões Disponíveis 15
5.6 Conclusão da Missão 16
5.7 Registrar Ofensiva 17
5.8 Tela do Avatar 18
5.9 Loja 19
5.10 Inventario 20
5.11 Perfil 21
6. Modelo de Entidade-Relacionamento (MER) **21**
7. Diagrama de Classes **22**
8. Procedimentos para Rodar a Aplicação **23**
8.1 Ferramentas Utilizadas 23
8.2 Passo a Passo 24
Importar o projeto no IntelliJ 24
Executar localmente 24
Versão publicada 25

1. Objetivo e Escopo do Projeto

O SoulAI incentiva hábitos sustentáveis por meio de gamificação: o usuário cumpre missões, ganha moedas, compra acessórios na loja e os equipa no seu avatar.

link do github
link do render

O endereço base da API não possui uma rota definida na raiz (`/`). Por isso, ao acessar o link diretamente no navegador, o servidor retornará a mensagem de erro padrão da aplicação. Isso **não significa que o deploy está quebrado**, apenas indica que a raiz não expõe um endpoint público.

Para verificar e testar o funcionamento da API publicada, utilize rotas válidas especificadas na tabela de endpoints descritas no documento.

1.1 Objetivo do Projeto

O objetivo do **SoulAI** é promover o engajamento do usuário em ações sustentáveis através de um ambiente simples, interativo e motivador. O sistema busca estimular a frequência de uso por meio da ofensiva diária e recompensar o usuário quando ele conclui missões relacionadas à sustentabilidade.

Além disso, entregar o back-end do SoulAI como uma API RESTful em Java com Spring Boot, que o aplicativo (front-end) consome para cadastro, login, missões, loja e inventário, com os dados persistidos em um banco Oracle.

1.2 Escopo do Projeto

Escopo desta entrega (Sprint 4).

- API REST com Spring Boot, organizada em camadas Resource (controllers), BO (regras de negócio) e DAO (acesso ao banco via JDBC).
- Sete recursos: usuário, administrador, avatar, missão, acessório, missões do usuário e inventário do usuário, todos com CRUD completo.
- Regras de negócio na aplicação: validação de dados, senha com hash BCrypt, crédito de moedas ao concluir missão, compra com verificação de saldo e um acessório equipado por tipo.

- CORS liberado para o front-end e tratamento de erros com códigos HTTP corretos.
- Publicação no Render, em `https://soulai-api.onrender.com`

O escopo não inclui autenticação por token nas rotas, nem endpoints para as tabelas de ofensiva, interação, conversa e ações sustentáveis, que existem no banco mas não são expostas pela API.

2. Solução proposta

A solução é uma API RESTful que recebe requisições HTTP do aplicativo SoulAI, aplica as regras do jogo e grava tudo no Oracle, devolvendo sempre JSON. Cada recurso do sistema vira uma URI (`/usuario`, `/missao`, `/acessorio`...) e as operações usam os verbos GET, POST, PUT e DELETE com os códigos de status adequados.

Camada	Pacote	Responsabilidade
Resource	`controller`	Recebe a requisição, chama a BO e monta a resposta HTTP. Inclui o `CorsFilter`.
BO	`bo`	Valida os dados e aplica as regras de negócio.
DAO	`dao`	Único lugar com SQL, via JDBC e `PreparedStatement`.
Model	`entities`	Classes que representam as tabelas.
Conexão	`conexoes`	`ConexaoFactory`, que abre a conexão com o Oracle.
Exceções	`excecoes`	Exceções próprias e o tratador global que as converte em códigos HTTP.

3. Funcionalidades

Todas as funcionalidades abaixo estão implementadas na API publicada. As regras que antes ficavam em triggers do banco (crédito de moedas e um acessório equipado por tipo) agora são executadas pela aplicação, dentro de transações.

Funcionalidade	O que faz	Regras principais
Cadastro e login	Cria a conta e autentica por e-mail e senha	E-mail válido e único, senha com no mínimo 6 caracteres guardada com hash BCrypt, mesma mensagem para e-mail ou senha errados
Perfil	Consulta e edita nome, e-mail, senha e foto	Saldo de moedas e ofensiva não são editados por aqui
Avatar	Cria e edita o avatar do usuário	Um avatar por usuário
Missões	Lista o catálogo e as missões ativas	Só missões ativas podem ser atribuídas
Missões do usuário	Atribui, atualiza o progresso e conclui	Progresso de 0 a 99. Concluir credita as moedas e grava a data em uma única transação, e só acontece uma vez por missão
Loja	Lista os acessórios com imagem e permite comprar	Exige saldo suficiente e item não repetido. Debita as moedas e adiciona ao inventário na mesma transação
Inventário	Lista os itens, equipa e desequipa	Um acessório equipado por tipo: equipar um item desequipa os outros do mesmo tipo

Administração	Administradores criam e editam missões e acessórios e fazem login	O administrador precisa existir. Itens já vinculados a usuários não podem ser removidos
Tratamento de erros	Toda falha devolve um JSON com `status` e `mensagem`	400, 401, 404, 409 e 500
Publicação	API disponível na internet para o front-end	Docker no Render, com CORS liberado

4. Tabela de endpoints

Endereço base: `https://soulai-api.onrender.com.` As colunas mostram o status de sucesso e os de erro mais prováveis. Qualquer rota pode devolver 500 se o banco falhar, e 400 se o JSON for malformado ou o id não for numérico.

Usuário

Verbo	URI	Sucesso	Erros
GET	`/usuario`	200	500
GET	`/usuario/{id}`	200	404
POST	`/usuario`	201	400, 409
PUT	`/usuario/{id}`	200	400, 404, 409

DELETE	`/usuario/{id}`	204	404, 409
POST	`/usuario/login`	200	400, 401

Administrador

Verbo	URI	Sucesso	Erros
GET	`/administrador`	200	500
GET	`/administrador/{id}`	200	404
POST	`/administrador`	201	400, 409
PUT	`/administrador/{id}`	200	400, 404, 409
DELETE	`/administrador/{id}`	204	404, 409
POST	`/administrador/login`	200	400, 401

Avatar

Verbo	URI	Sucesso	Erros
GET	`/avatar`	200	500

GET	`/avatar/{id}`	200	404
GET	`/avatar/usuario/{idUsuario}`	200	404
POST	`/avatar`	201	400, 404, 409
PUT	`/avatar/{id}`	200	400, 404
DELETE	`/avatar/{id}`	204	404

Acessório

Verbo	URI	Sucesso	Erros
GET	`/acessorio`	200	500
GET	`/acessorio/{id}`	200	404
POST	`/acessorio`	201	400, 404
PUT	`/acessorio/{id}`	200	400, 404
DELETE	`/acessorio/{id}`	204	404, 409

Missão

Verbo	URI	Sucesso	Erros
GET	`/missao`	200	500
GET	`/missao/ativas`	200	500
GET	`/missao/{id}`	200	404
POST	`/missao`	201	400, 404
PUT	`/missao/{id}`	200	400, 404
DELETE	`/missao/{id}`	204	404, 409

Missões do usuário

Verbo	URI	Sucesso	Erros
GET	`/usuario/{idUsuario}/missoes`	200	404

GET	`/usuario/{idUsuario}/missoes/{idMissao}`	200	404
POST	`/usuario/{idUsuario}/missoes/{idMissao}`	201	400, 404, 409
PUT	`/usuario/{idUsuario}/missoes/{idMissao}/progresso`	200	400, 404, 409
POST	`/usuario/{idUsuario}/missoes/{idMissao}/concluir`	200	400, 404, 409
DELETE	`/usuario/{idUsuario}/missoes/{idMissao}`	204	404, 409

Inventário do usuário

Verbo	URI	Sucesso	Erros
GET	`/usuario/{idUsuario}/inventario`	200	404
GET	`/usuario/{idUsuario}/inventario/{idAcessorio}`	200	404
POST	`/usuario/{idUsuario}/inventario/{idAcessorio}`	201	400, 404, 409
PUT	`/usuario/{idUsuario}/inventario/{idAcessorio}/equipar`	200	404

PUT	`/usuario/{idUsuario}/inventario/{idAcessorio}/desequipar`	200	404
DELETE	`/usuario/{idUsuario}/inventario/{idAcessorio}`	204	404

5.Protótipo do Sistema

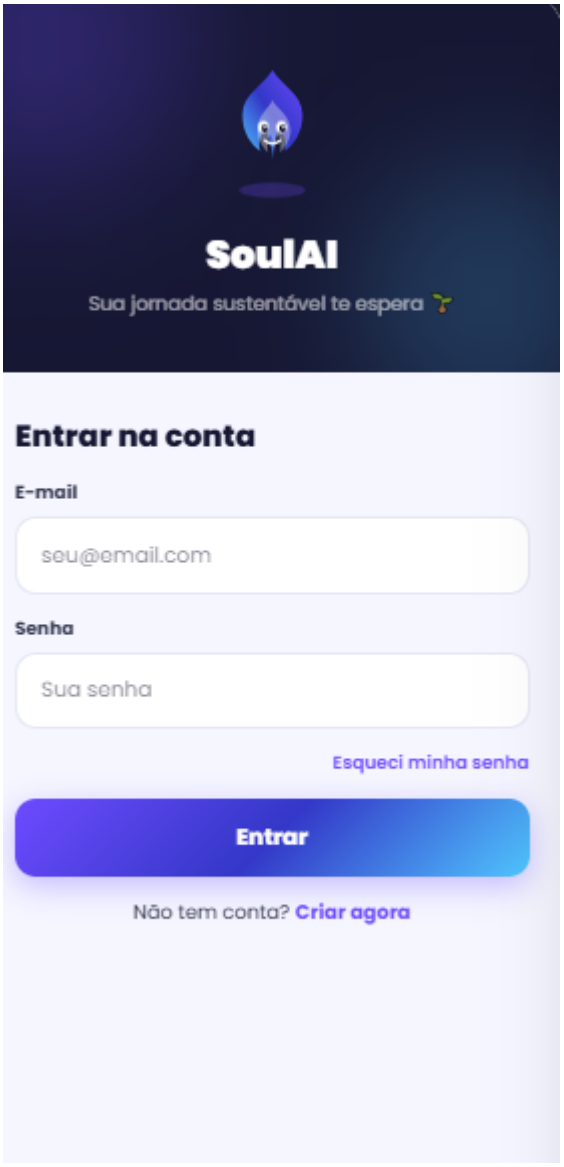
5.1 Tela Inicial



5.2 Tela de Cadastro



5.3 Tela de Login



5.4 Menu Principal



5.5 Tela de Missões Disponíveis



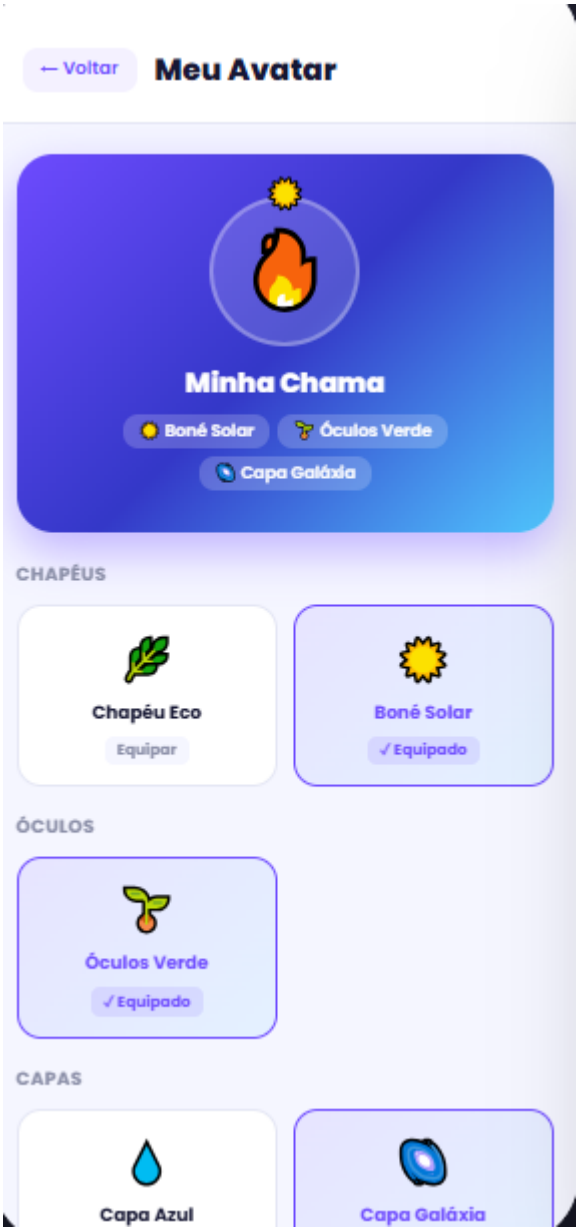
5.6 Conclusão da Missão



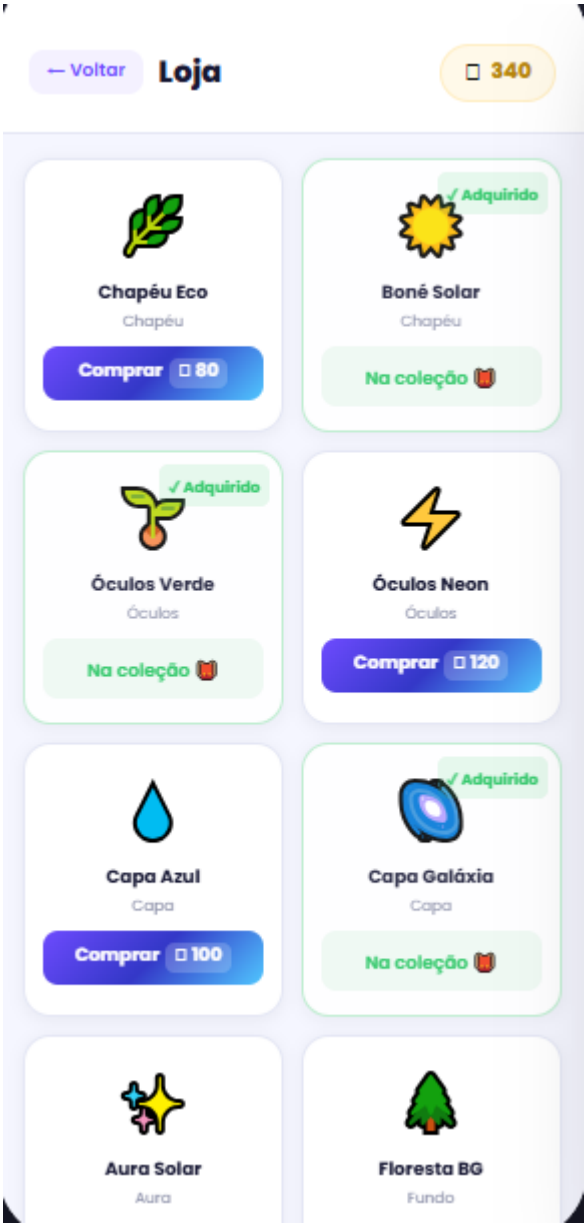
5.7 Registrar Ofensiva



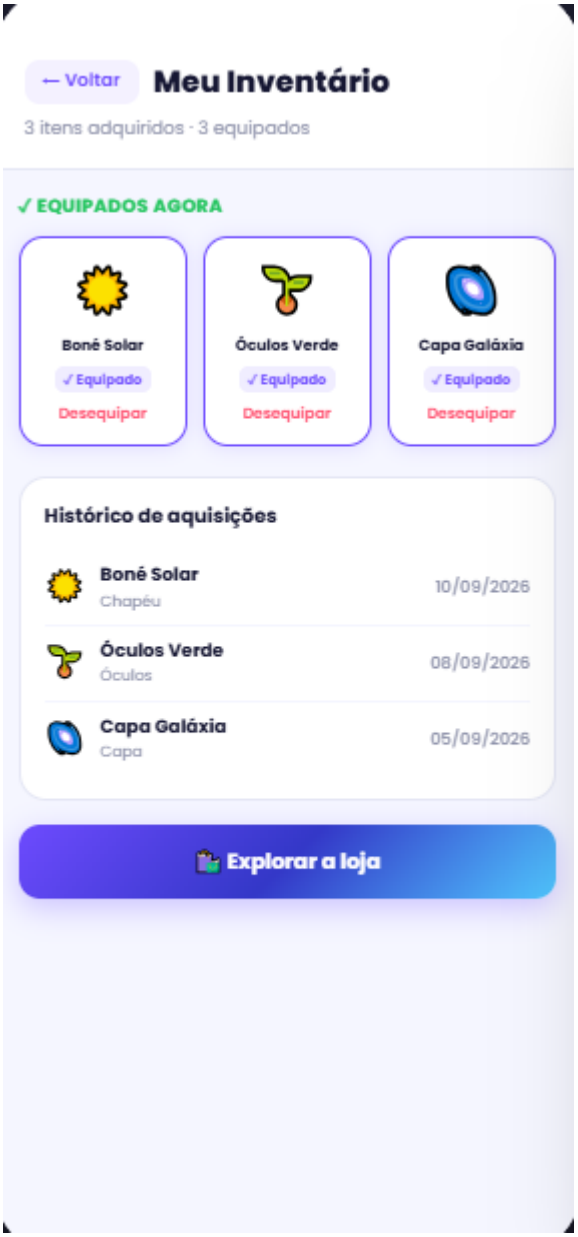
5.8 Tela do Avatar



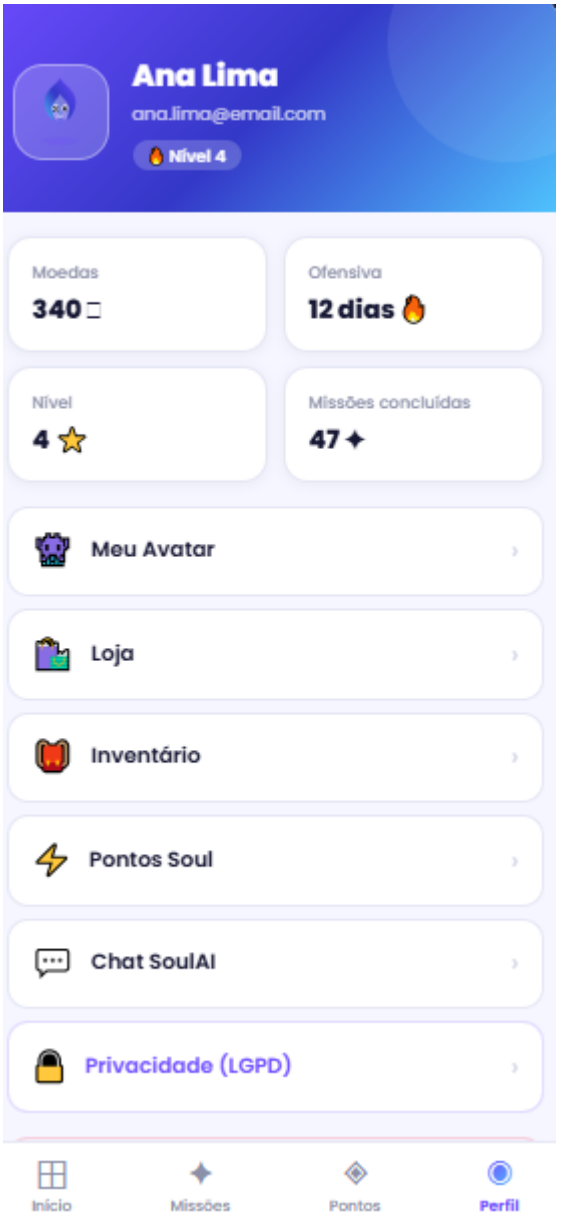
5.9 Loja


← Voltar
Loja
340
Chapéu Eco
Chapéu
Comprar 80
✓ Adquirido
Boné Solar
Chapéu
Na coleção
✓ Adquirido
Óculos Verde
Óculos
Na coleção
Óculos Neon
Óculos
Comprar 120
Capa Azul
Capa
Comprar 100
✓ Adquirido
Capa Galáxia
Capa
Na coleção
Aura Solar
Aura
Floresta BG
Fundo

5.10 Inventario



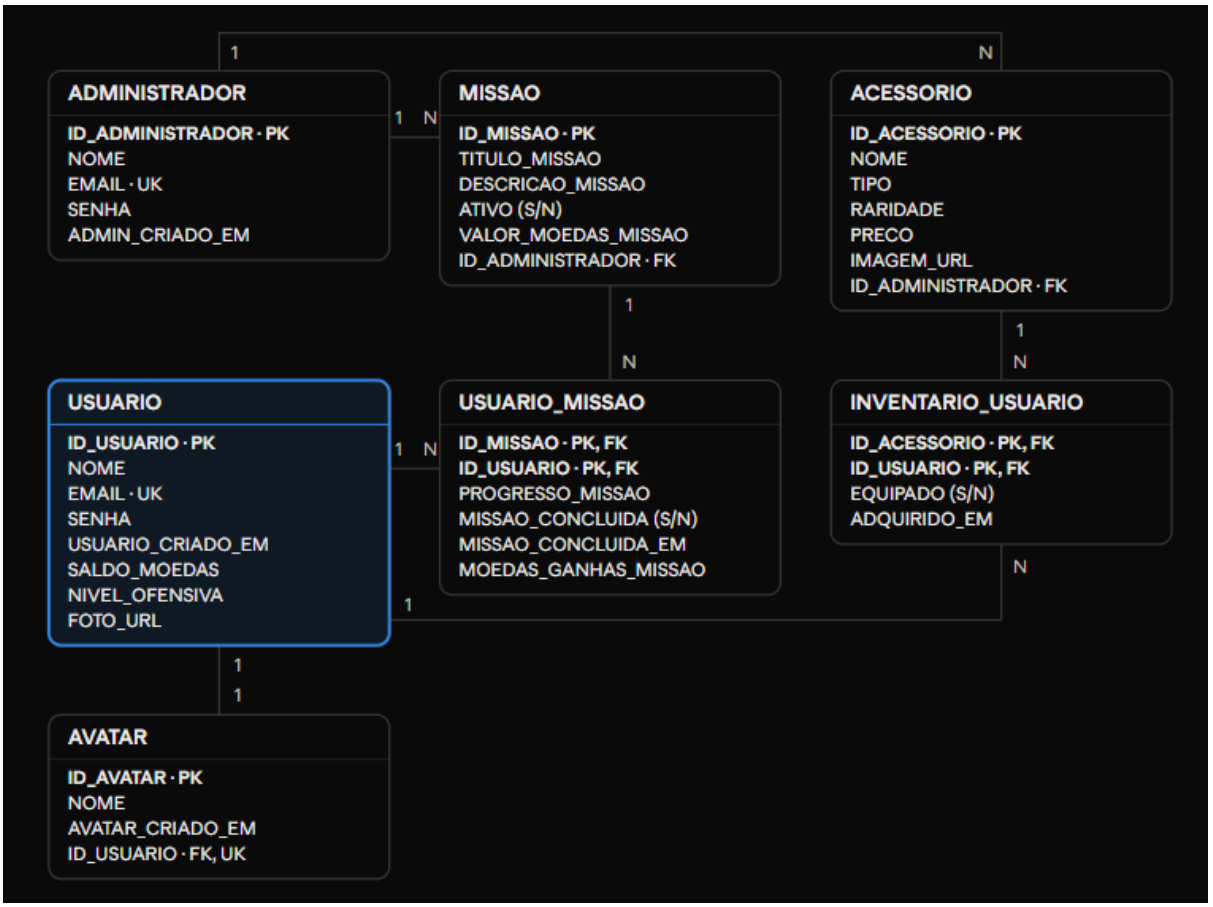
5.11 Perfil



6. Modelo de Entidade-Relacionamento (MER)

Um administrador cadastra várias missões e acessórios. O usuário tem um único avatar e se liga a missões e acessórios por tabelas associativas, cuja chave primária é composta pelas duas chaves estrangeiras. O banco tem outras nove tabelas (ofensiva, interação, conversa, mensagens e ações sustentáveis), que ficam fora desta entrega porque a API ainda não as expõe. As colunas `FOTO_URL` e `IMAGEM_URL` são `VARCHAR2(500)`.

Regras de exclusão: o avatar é apagado junto com o usuário (`ON DELETE CASCADE`). Os demais vínculos bloqueiam a exclusão, e a API devolve 409.

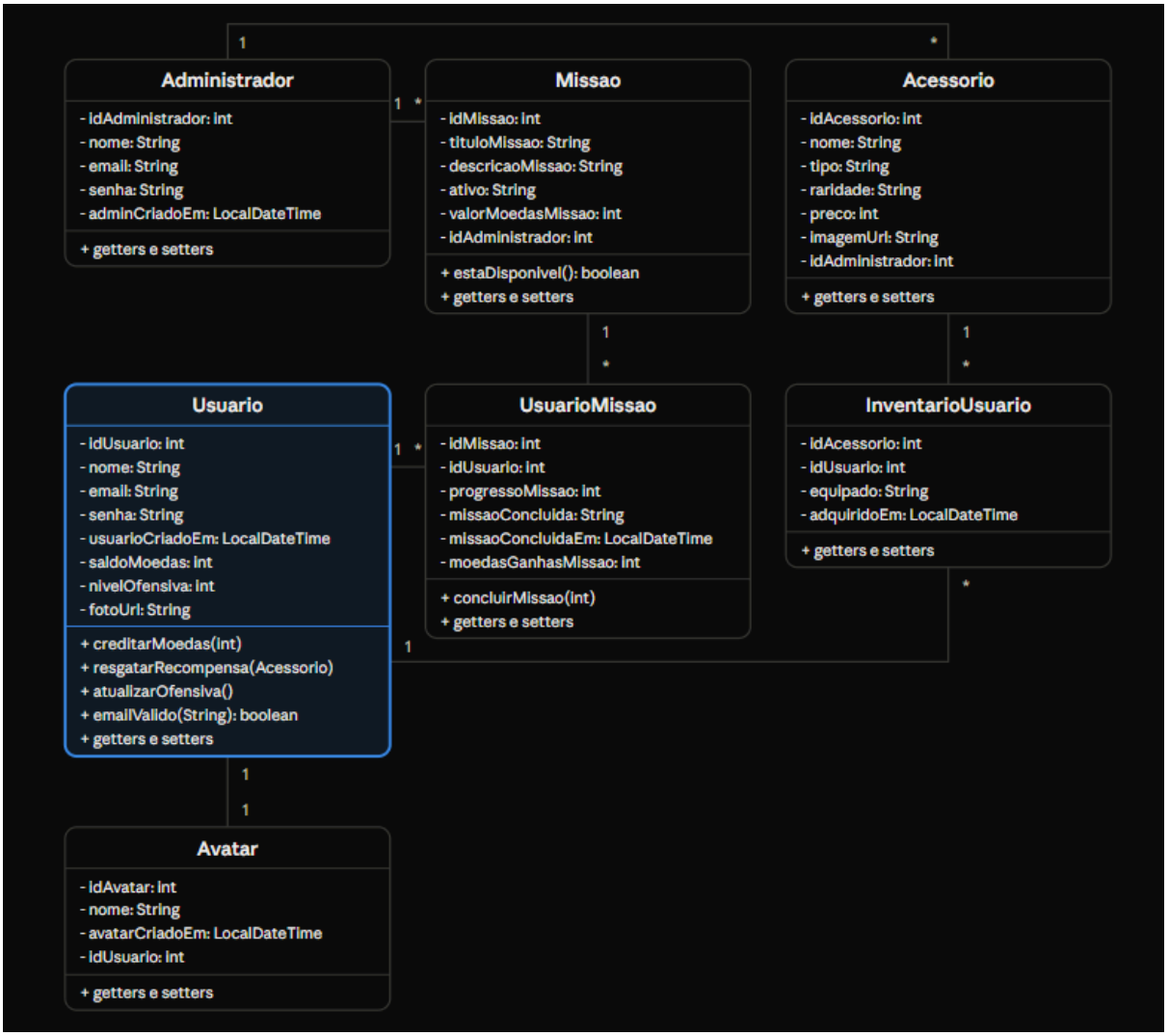


7. Diagrama de Classes

Cada classe do pacote `entities` corresponde a uma tabela do MER. As regras de negócio estão em `Usuario` (`creditarMoedas`, `resgatarRecompensa`, `atualizarOfensiva` e o método estático `emailValido`), em `Missao` (`estaDisponivel`) e em `UsuarioMissao` (`concluirMissao`). As demais classes são de dados, com construtores, getters e setters.

O fluxo de cada requisição atravessa as camadas `controller` (Resource), `bo` e `dao`, nessa ordem, e só o `dao` fala com o Oracle por meio da `ConexaoFactory`. Os erros de negócio viram respostas HTTP pelo `GlobalExceptionHandler` (`RecursoNaoEncontradoException` → 404, `ValidacaoException` → 400, `NaoAutorizadoException` → 401, `ConflitoException` → 409).

.



8. Procedimentos para Rodar a Aplicação

8.1 Ferramentas Utilizadas

Ferramenta	Versão	Uso
JDK	21	Compilar e executar a API
Maven	Wrapper do projeto (`mvnw`), sem instalação própria	Build e dependências

Spring Boot	4.1.1	Framework da API
IntelliJ IDEA	Community ou Ultimate	IDE recomendada
Oracle SQL Developer	Qualquer versão recente	Executar os scripts do banco
Insomnia ou Postman	Qualquer versão recente	Testar os endpoints
Docker	Opcional	Gerar a imagem usada no deploy

8.2 Passo a Passo

Importar o projeto no IntelliJ

1. Clone o repositório do GitHub ou baixe o ZIP e extraia.
2. No IntelliJ, escolha **File** → **Open** e selecione a pasta que contém o `pom.xml`. Abrir só o arquivo ou uma pasta acima causa erro de importação do Maven.
3. Se perguntado, confie no projeto e aguarde o Maven baixar as dependências.
4. Em **File** → **Project Structure** → **Project**, confirme o SDK 21.

Executar localmente

1. Abra a classe `SoulAiApplication` e clique em **Run**.
2. Aguarde a mensagem `Started SoulAiApplication`.
3. Teste no Insomnia com `GET http://localhost:8080/missao/ativas`. Deve voltar 200 com a lista de missões.
4. Para um teste completo, faça `POST /usuario`, depois `POST /usuario/login`.

Versão publicada

A API está publicada no Render em `https://soulai-api.onrender.com`, a partir do `Dockerfile` do repositório. No plano gratuito o serviço hiberna e a primeira requisição pode levar cerca de 1 minuto. Faça uma chamada de aquecimento antes de usar.